

A HANDS-ON, PORTFOLIO-GRADE TUTORIAL

Maximizing PostgreSQL for Machine Learning Portfolio Projects

From raw tables to a published GitHub repository and a live Power BI front end — statistics, in-database ML algorithms, feature pipelines, BI export, and the storytelling that gets you hired.

80 Slides • 9 Sections • Design your own end-to-end data project



What We're Covering, Section by Section

01 Why Portfolio Projects & GitHub Matter

02 PostgreSQL Foundations for ML

03 Statistical Algorithms in SQL

04 ML Algorithms Inside PostgreSQL

05 Feature Engineering Pipelines

06 Performance & Scaling for ML Workloads

07 Exporting PostgreSQL to Power BI

08 Designing the Front-End Demo

09 Publishing the Project on GitHub

01

SECTION 1 OF 9

Why Portfolio Projects & GitHub Matter

Before we touch SQL, we need to understand why this entire exercise exists: a portfolio project is the single highest-leverage artifact you can put in front of a hiring manager.



Why Portfolios Outperform Resumes

A resume claims skill. A portfolio proves it.



Resumes are claims

“Proficient in SQL and Power BI” is unverifiable and identical on a thousand other resumes.



Iterative signal

Commit history shows how you think over time, not just a polished final snapshot.



Portfolios are evidence

A working repo shows real schema design, real queries, and a real dashboard — no interpretation needed.



Differentiation

Most candidates submit the same tutorial dataset with no original analysis — depth is what stands out.

What Hiring Managers Actually Check on GitHub



The 90-second scan

README, pinned repos, and commit graph are read before a single line of code.



Commit quality

Frequent, descriptive commits beat one giant “final upload” commit every time.



Project structure

Clear folders (data / sql / notebooks / dashboards / docs) signal engineering discipline.



Evidence of outcome

Screenshots of the Power BI dashboard and a plain-English summary of findings.

Anatomy of a Standout GitHub Repository

What belongs in the repo

- A README with problem, data, method, and results
- /sql — schema, seed data, algorithm queries
- /pipeline — feature engineering & ETL scripts
- /dashboard — .pbix file or exported screenshots
- /docs — ERD diagram and architecture notes
- LICENSE and .gitignore

Signals this sends

- You can structure a real production project
- You document decisions, not just deliver code
- You understand the full pipeline, not one step
- You know how to separate concerns cleanly
- You're reusable on day one of the job
- You respect licensing and collaboration norms

Portfolio Project Checklist

Do This

- ✓ Pick a dataset with a real, specific question to answer
- ✓ Write SQL that demonstrates reasoning, not just SELECT *
- ✓ Explain trade-offs in the README (why Postgres, why this model)
- ✓ Ship a working, screenshot-documented dashboard

Avoid This

- ✗ Uploading a single Jupyter notebook with no structure
- ✗ Copy-pasting a tutorial dataset with zero original angle
- ✗ Leaving commit messages as “update” or “fix”
- ✗ Shipping a dashboard with no narrative or takeaway

THE CASE STUDY

What We'll Build Across This Tutorial

One continuous pipeline, from raw rows to an interview-ready demo.

1

PostgreSQL database with a
realistic customer / transactions
schema

12+

In-database algorithms —
statistics, regression, clustering,
similarity search

1

Power BI dashboard connected
live via DirectQuery / Import

1

Public GitHub repository, fully
documented and interview-ready

Every technique from here forward maps directly onto this one project — you can follow along with your own dataset.

02

SECTION 2 OF 9

PostgreSQL Foundations for ML

Before running algorithms, we need the right foundation: why Postgres, how it's built, what to install, and how to shape a schema that ML workloads actually want.



Why PostgreSQL for ML Workloads

Postgres Strengths

- ✓ Native window functions & statistical aggregates
- ✓ pgvector brings embeddings + similarity search in-DB
- ✓ PL/Python & PL/R let you run real model code near the data
- ✓ Free, open-source, and trivially portable in a repo

Where It's Not the Tool

- ✗ Training deep neural networks at scale
- ✗ Massive distributed training across GPU clusters
- ✗ Real-time inference at sub-millisecond latency
- ✗ Replacing a dedicated feature-store platform at big scale

Takeaway: Postgres is the ideal engine for analytical ML, classic algorithms, and feature pipelines — exactly what a portfolio project needs to demonstrate.

How a Query Becomes an ML-Ready Result

1 Client (psql / Power BI / Python)

↓ SQL over libpq

2 Parser → Planner → Optimizer

↓ chosen execution plan

Executor (scans, joins, aggregates)

↓ reads/writes

3 Storage Engine + WAL + Extensions

1

Any client that speaks the Postgres wire protocol can run your ML queries — including Power BI.

2

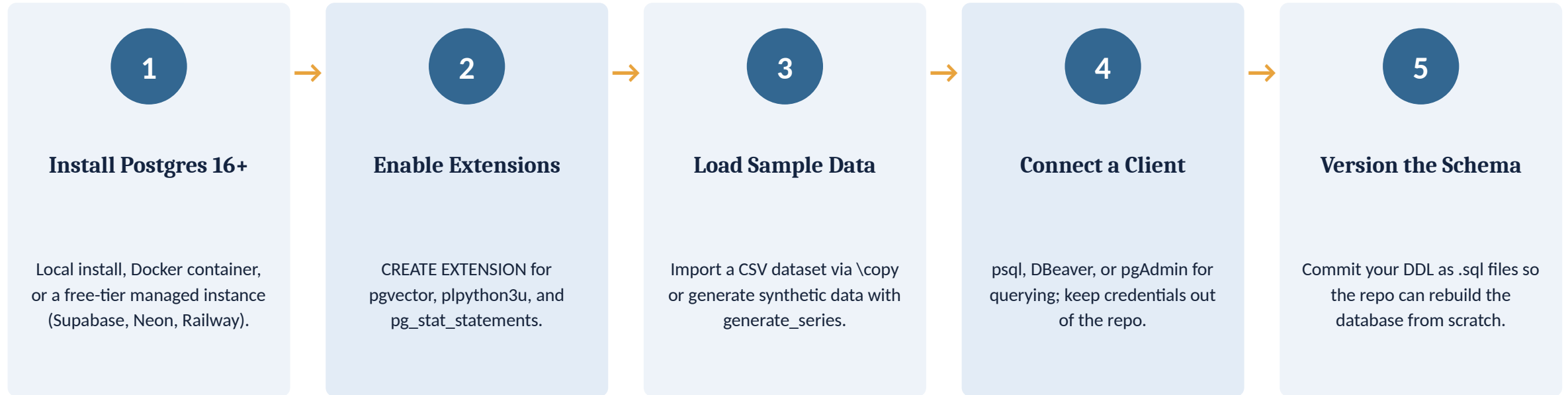
The planner picks the fastest join/scan strategy — this is what EXPLAIN ANALYZE shows you.

3

Extensions like pgvector and PL/Python plug directly into the executor, not a separate service.

→ One engine handles storage, querying, and ML computation — no separate service to deploy for a portfolio project.

Building Your ML-Ready Postgres Environment



Key Extensions for In-Database Machine Learning



pgvector

Stores embeddings and enables cosine / L2 similarity search directly in SQL.



PL/Python3u

Write real Python (NumPy, scikit-learn) inside a Postgres function.



MADlib

A mature in-database library of regression, clustering, and classification algorithms.



TimescaleDB

Time-series superpowers — continuous aggregates, hypertables, gap filling.



PostGIS

Geospatial analysis — distance, clustering, and routing for location-based features.



pg_cron

Schedules recurring feature-refresh and model-retraining jobs inside the DB itself.

Data Types That Make ML Easier in Postgres

Type → ML Use Case

- NUMERIC — exact precision for financial/statistical features
- JSONB — flexible raw event or API payload storage
- ARRAY — store per-row feature vectors compactly
- VECTOR (pgvector) — embeddings for similarity search
- TIMESTAMPTZ — correct time-zone-safe time-series joins

Practical Guidance

- Prefer NUMERIC over FLOAT for anything aggregated repeatedly
- Index JSONB fields you filter on with a GIN index
- Normalize arrays into rows before joining across tables
- Keep vector dimensionality consistent within a column
- Store raw timestamps in UTC; convert at the BI layer

Indexing Strategies for ML Query Patterns

```
-- B-tree: equality & range lookups
1 CREATE INDEX ix_txn_date ON txns(txn_date);

-- GIN: JSONB & full-text features
2 CREATE INDEX ix_evt ON events USING GIN(payload);

-- BRIN: huge, naturally-ordered tables
3 CREATE INDEX ix_log_ts ON logs USING BRIN(ts);
```

1

B-tree is the default — use it for dates, IDs, and any WHERE/ORDER BY column.

2

GIN indexes make JSONB containment queries and feature-flag filters fast.

3

BRIN is tiny and ideal for append-only, time-ordered event or log tables.



Right-sizing the index type keeps feature-extraction queries fast even as your training table grows.

Schema Design: The Feature-Store Pattern

Raw / Staging Layer

- raw_transactions — untouched source rows
- raw_customers — append-only ingestion
- raw_events — JSONB payloads from apps/APIs
- No transformation logic lives here

Feature / Serving Layer

- fct_customer_features — one row per entity
- Built with materialized views, refreshed on schedule
- Indexed for the exact filters your model/BI tool uses
- This layer is what Power BI and your models query



03

SECTION 3 OF 9

Statistical Algorithms in SQL

Every ML pipeline starts with description before prediction. These are the statistical building blocks that turn raw rows into understood data — written natively in SQL.



Descriptive Statistics with Aggregate Functions

```
SELECT
```

```
1  AVG(order_value)      AS mean_val,  
2  STDDEV_SAMP(order_value) AS std_dev,  
3  VARIANCE(order_value) AS variance,  
   MIN(order_value), MAX(order_value)  
FROM orders;
```

1 AVG gives the central tendency — your baseline feature for every numeric column.

2 STDDEV_SAMP measures spread; large values often flag features worth scaling.

3 VARIANCE feeds directly into z-score and outlier-detection queries later.

→ One aggregate query gives you the profile every EDA notebook opens with — no Python required.

Percentiles & Distribution Analysis

```
SELECT  
1 PERCENTILE_CONT(0.5)  
  WITHIN GROUP (ORDER BY order_value) AS median,  
2 PERCENTILE_CONT(0.90)  
  WITHIN GROUP (ORDER BY order_value) AS p90  
3 FROM orders;
```

1 PERCENTILE_CONT interpolates a continuous value — ideal for medians on skewed revenue data.

2 The 90th percentile is a standard feature for flagging high-value customers.

3 Swap CONT for DISC to snap to an actual observed row instead of interpolating.

→ Percentiles reveal skew that a mean alone hides — critical before choosing a scaling strategy.

Correlation Analysis Between Features

```
SELECT
```

```
1 CORR(session_minutes, order_value) AS r,  
2 COVAR_POP(session_minutes, order_value) AS cov  
FROM customer_sessions;
```

1

CORR returns Pearson's r between -1 and 1 — fast feature-relevance screening.

2

COVAR_POP is the raw covariance behind CORR, useful when building your own regression math.



Correlation screening in SQL lets you drop redundant features before any model touches the data.

Window Functions for Trend Analysis

```
SELECT customer_id, order_date, order_value,  
1  RANK() OVER (PARTITION BY customer_id  
    ORDER BY order_value DESC) AS value_rank,  
2  LAG(order_value) OVER (PARTITION BY customer_id  
    ORDER BY order_date) AS prev_value  
FROM orders;
```

1 PARTITION BY resets the calculation per customer — the SQL equivalent of a groupby-apply.

2 LAG pulls the previous row's value, the basis for change-over-time and churn features.

→ Window functions compute per-entity trends without collapsing the row-level detail you still need.

Moving Averages & Time-Series Smoothing

```
SELECT order_date,  
       AVG(daily_revenue) OVER (  
         ORDER BY order_date  
1      ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
2      ) AS rolling_7d_avg  
FROM daily_sales;
```

1

The frame clause defines the rolling window — here, a trailing 7-day average.

2

Smoothing removes day-to-day noise so trend features are stable for a model or chart.



This single pattern powers the smoothed trend line every BI dashboard needs for time-series data.

Z-Score Outlier Detection

```
1 WITH stats AS (  
    SELECT AVG(order_value) m, STDDEV(order_value) s  
    FROM orders)  
2 SELECT o.*, (o.order_value - s.m) / s.s AS z_score  
    FROM orders o, stats s  
3 WHERE ABS((o.order_value - s.m) / s.s) > 3;
```

1

A CTE computes the population mean and std-dev once, reused for every row.

2

The z-score formula: how many standard deviations a value sits from the mean.

3

$|z| > 3$ is the classic statistical threshold for flagging an anomaly.



Pure-SQL anomaly detection — no external library needed to flag suspicious transactions.

IQR-Based Outlier Detection

```
WITH q AS (
```

```
1 SELECT PERCENTILE_CONT(0.25) WITHIN GROUP  
    (ORDER BY order_value) AS q1,  
    PERCENTILE_CONT(0.75) WITHIN GROUP  
    (ORDER BY order_value) AS q3  
FROM orders)
```

```
SELECT * FROM orders, q  
2 WHERE order_value NOT BETWEEN  
    q1 - 1.5*(q3-q1) AND q3 + 1.5*(q3-q1);
```

1 Q1 and Q3 define the interquartile range — more robust to skew than z-scores.

2 The 1.5×IQR rule is the same fence used to draw box-plot whiskers.

→ IQR-based filtering is the safer default when your revenue or usage data isn't normally distributed.

Recursive CTEs for Hierarchical & Graph Algorithms

```
WITH RECURSIVE org_chart AS (  
  1 SELECT id, manager_id, 1 AS depth  
    FROM employees WHERE manager_id IS NULL  
  2 UNION ALL  
    SELECT e.id, e.manager_id, oc.depth + 1  
    FROM employees e  
  3 JOIN org_chart oc ON e.manager_id = oc.id)  
SELECT * FROM org_chart;
```

1 The anchor query defines the root nodes — here, employees with no manager.

2 UNION ALL repeatedly joins new rows until no further matches exist.

3 This same pattern powers referral-network and bill-of-materials traversal.

→ Recursive CTEs implement graph-traversal logic that would otherwise require a separate graph library.

generate_series for Simulation & Synthetic Data

```
SELECT
1  gs::date AS day,
2  round((random()*500 + 100)::numeric, 2) AS revenue
FROM generate_series(
    '2025-01-01'::date, '2025-12-31'::date,
    interval '1 day') AS gs;
```

1 generate_series produces one row per day — perfect for filling calendar gaps.

2 Combined with random(), it builds realistic synthetic datasets for a portfolio demo.

→ No dataset? generate_series lets you build a believable, reproducible one in a single query.

04

SECTION 4 OF 9

ML Algorithms Inside PostgreSQL

The centerpiece of the tutorial: implementing real machine-learning algorithms — regression, clustering, classification, similarity search — natively inside Postgres.



Simple Linear Regression (regr_slope / regr_intercept)

```
SELECT
```

```
1  regr_slope(order_value, session_minutes) AS slope,  
    regr_intercept(order_value, session_minutes) AS b0,  
2  regr_r2(order_value, session_minutes) AS r_squared  
FROM customer_sessions;
```

1 regr_slope computes the least-squares slope — the full model fit in one aggregate.

2 regr_r2 tells you how much variance the line explains — your model-quality metric.

→ Postgres ships ordinary least-squares regression as a built-in aggregate — zero extensions required.

Logistic Regression Scoring in PL/pgSQL

```
CREATE FUNCTION predict_churn(minutes numeric)
RETURNS numeric AS $$
1 SELECT 1 / (1 + exp(-(
2   -1.2 + 0.03 * minutes
   )));
$$ LANGUAGE sql IMMUTABLE;
```

1

The sigmoid function $1/(1+e^{-x})$ converts a linear score into a 0-1 probability.

2

Coefficients (here -1.2, 0.03) come from a model trained offline in scikit-learn or MADlib.



Once trained, logistic regression scoring is just a function call — fast, live inference inside SQL.

K-Means Clustering with PL/Python

```
CREATE FUNCTION kmeans_segments(k int)
RETURNS TABLE(customer_id int, cluster int)
AS $$
```

```
1 from sklearn.cluster import KMeans
2 rows = plpy.execute(
    'SELECT id, recency, frequency FROM rfm')
    km = KMeans(n_clusters=k).fit(...)
$$ LANGUAGE plpython3u;
```

1

plpython3u gives you full scikit-learn inside a Postgres function — no external service.

2

plpy.execute() pulls rows straight from the table, avoiding any data export step.



Customer segmentation (RFM clustering) can be trained and served without ever leaving the database.

K-Nearest Neighbors with pgvector

```
SELECT customer_id, embedding
1  <-> (SELECT embedding FROM customers
        WHERE customer_id = 42) AS distance
FROM customers
2 ORDER BY distance
LIMIT 5;
```

1 The <-> operator is pgvector's distance operator — L2 by default.

2 ORDER BY distance + LIMIT is literally the K in K-Nearest Neighbors.

→ “Find similar customers” becomes a five-line query once embeddings live in a vector column.

Naive Bayes Classification in SQL

```
SELECT class,  
1  PRODUCT(likelihood) * prior AS posterior  
FROM (  
    SELECT class, prior,  
           feature_prob(feature, value, class) AS likelihood  
    FROM training_features) t  
2 GROUP BY class, prior  
ORDER BY posterior DESC;
```

1

Naive Bayes multiplies per-feature likelihoods by the class prior — pure aggregation logic.

2

The highest posterior probability across classes is the predicted label.



A probabilistic classifier reduces to GROUP BY + multiplication — elegant, auditable SQL.

Decision-Tree Logic with CASE & Recursive Rules

```
SELECT customer_id,  
1 CASE  
    WHEN recency < 30 AND frequency > 5  
        THEN 'high_value'  
    WHEN recency < 90  
        THEN 'active'  
    ELSE 'at_risk'  
2 END AS segment  
FROM rfm_features;
```

1 Each WHEN branch mirrors a trained decision-tree split — exported as thresholds.

2 A tree trained in Python can be exported directly into this exact CASE structure.

→ Once a decision tree is trained, its logic deploys as a readable, dependency-free CASE expression.

Market-Basket / Association Rule Mining

```
SELECT a.product_id AS item_a, b.product_id AS item_b,  
       COUNT(*)::numeric / (  
         SELECT COUNT(DISTINCT order_id) FROM order_items  
       ) AS support  
FROM order_items a  
1 JOIN order_items b  
   ON a.order_id = b.order_id AND a.product_id < b.product_id  
GROUP BY 1, 2  
2 HAVING COUNT(*) > 10;
```

1 Self-joining order_items on order_id finds every co-purchased product pair.

2 The HAVING clause is your minimum-support threshold — the core Apriori concept.

→ “Customers who bought X also bought Y” is a self-join and a GROUP BY — no external library needed.

Anomaly Detection with Rolling Statistics

```
SELECT ts, value,  
       AVG(value) OVER w AS trail_avg,  
       STDDEV(value) OVER w AS trail_std,  
2  (value - AVG(value) OVER w)  
   / NULLIF(STDDEV(value) OVER w,0) AS z  
FROM sensor_readings  
1 WINDOW w AS (ORDER BY ts  
               ROWS BETWEEN 20 PRECEDING AND 1 PRECEDING);
```

1

The WINDOW clause defines a trailing 20-reading baseline — no future data leaks in.

2

Each point is scored against its own recent history, ideal for streaming metrics.



This pattern flags sensor or traffic anomalies in real time as rows are inserted.

Vector Embeddings & Similarity Search

```
CREATE TABLE product_embeddings (  
    product_id int PRIMARY KEY,  
    1 embedding vector(384)  
);  
2 CREATE INDEX ON product_embeddings  
    USING ivfflat (embedding vector_cosine_ops);
```

1 vector(384) matches common sentence-embedding model output dimensions.

2 An ivfflat index makes approximate nearest-neighbor search fast at scale.

→ This table is the foundation for semantic search, recommendations, and RAG-style lookups in Postgres.

Cosine Similarity for Semantic Search

```
SELECT product_id,  
1 1 - (embedding <=> :query_embedding) AS similarity  
FROM product_embeddings  
2 ORDER BY embedding <=> :query_embedding  
LIMIT 10;
```

1

The <=> operator returns cosine distance; 1 minus it gives similarity from 0 to 1.

2

Sorting by distance ascending surfaces the most semantically similar rows first.



This exact query pattern powers “search by meaning” features in modern AI-driven apps.

MADlib & PostgresML: Full In-Database ML

MADlib

- Mature, SQL-callable library of ML algorithms
- `linregr_train()`, `kmeans`, decision trees, SVMs
- Battle-tested at large scale (Pivotal / Greenplum lineage)
- Best for classic statistical & regression models

PostgresML

- Trains scikit-learn / XGBoost models via SQL functions
- `pgml.train()` and `pgml.predict()` as first-class SQL calls
- Supports embeddings & LLM inference in-database too
- Best when you want modern models without leaving SQL

05

SECTION 5 OF 9

Feature Engineering Pipelines

Algorithms are only as good as their inputs. This section builds the SQL pipeline that turns raw transactional data into clean, model-ready features.



Feature Engineering Patterns in SQL



Aggregation features

COUNT, SUM, AVG per entity — total orders, lifetime value, average basket size.



Ratio features

Cross-metric ratios — return rate, discount rate, sessions per order.



Recency features

DATE differences from a reference point — days since last purchase or login.



Lag / trend features

Window functions comparing current vs prior period behavior.

Encoding Categorical Variables in SQL

```
SELECT customer_id,  
1 CASE WHEN plan='gold' THEN 1 ELSE 0 END AS is_gold,  
CASE WHEN plan='silver' THEN 1 ELSE 0 END AS is_silver,  
2 CASE WHEN plan='bronze' THEN 1 ELSE 0 END AS is_bronze  
FROM customers;
```

1

Each CASE column is a manual one-hot encode — no library needed for low-cardinality fields.

2

For high-cardinality categories, prefer target or frequency encoding instead of one-hot.



One-hot encoding in pure SQL keeps the entire feature pipeline reproducible from a single script.

Normalization & Scaling in SQL

```
SELECT customer_id,  
1 (order_value - MIN(order_value) OVER ())  
  / NULLIF(MAX(order_value) OVER ()  
    - MIN(order_value) OVER (), 0) AS value_minmax,  
2 (order_value - AVG(order_value) OVER ())  
  / NULLIF(STDDEV(order_value) OVER (), 0) AS value_z  
FROM orders;
```

1 Min-max scaling rescales values into a fixed 0–1 range for distance-based algorithms.

2 Z-score standardization centers on 0 with unit variance — the default for regression.

→ Both standard scaling methods are window functions — computed once, reused everywhere downstream.

Handling Missing Data in SQL

```
SELECT customer_id,  
1 COALESCE(age, (SELECT AVG(age) FROM customers)) AS  
age_imputed,  
2 CASE WHEN age IS NULL THEN 1 ELSE 0 END AS age_was_missing  
FROM customers;
```

1 COALESCE substitutes a sensible default — here, the population mean age.

2 A missing-value flag preserves the signal that imputation would otherwise hide.

→ Always pair imputation with a flag column — “missingness” is often predictive on its own.

Train / Test Split via SQL

```
SELECT *,  
1 CASE WHEN mod(customer_id, 5) = 0  
2 THEN 'test' ELSE 'train' END AS split  
FROM customer_features;
```

1

Using a deterministic modulo on the ID gives a reproducible, seed-free 80/20 split.

2

For time-series data, split by date instead of ID to avoid leaking the future.



A reproducible split column belongs in the feature table itself — not recomputed randomly each run.

Materialized Views as a Lightweight Feature Store

```
1 CREATE MATERIALIZED VIEW fct_customer_features AS  
  
  SELECT customer_id,  
         COUNT(*) AS order_count,  
         AVG(order_value) AS avg_order_value,  
         MAX(order_date) AS last_order_date  
  FROM orders GROUP BY customer_id;  
  
2 REFRESH MATERIALIZED VIEW CONCURRENTLY  
   fct_customer_features;
```

1

A materialized view pre-computes and stores the query result physically on disk.

2

CONCURRENTLY refreshes without locking readers — crucial for a live BI dashboard.

→ This is the single most useful pattern for a portfolio project: one queryable table that feeds both models and Power BI.

Scheduling Pipelines with pg_cron

```
SELECT cron.schedule(  
  'refresh_features',  
  1 '0 * * * *',  
  2 $$REFRESH MATERIALIZED VIEW CONCURRENTLY  
    fct_customer_features$$  
);
```

1

Standard cron syntax — here, the feature table refreshes every hour on the hour.

2

The scheduled body is just SQL — no separate orchestration service required.



pg_cron turns your feature pipeline from a manual script into a self-refreshing production system.

Data Quality & Validation Checks

Checks to Automate

- Row counts match expected source volumes
- No NULLs in required join keys
- Value ranges stay within plausible bounds
- Referential integrity between fact and dimension tables

Implementation

- CHECK constraints enforce ranges at write time
- FOREIGN KEY constraints enforce referential integrity
- A validation SQL script run before every dashboard refresh
- Log results to a data_quality_log table for auditability

06

SECTION 6 OF 9

Performance & Scaling for ML Workloads

ML queries scan large tables and compute heavy aggregates. These techniques keep the pipeline fast as your dataset grows from a demo to production scale.



Query Optimization with EXPLAIN ANALYZE

```
2 EXPLAIN (ANALYZE, BUFFERS)
  SELECT customer_id, AVG(order_value)
  FROM orders
  WHERE order_date > '2025-01-01'
  GROUP BY customer_id;
```

1 ANALYZE actually runs the query and reports real timings, not just estimates.

2 BUFFERS shows disk vs cache hits — the first thing to check when a query is slow.

→ Run EXPLAIN ANALYZE before and after every index change — it's the ground truth for performance work.

Partitioning Large ML Datasets

```
CREATE TABLE orders (  
    order_id bigint, order_date date, order_value numeric  
1) PARTITION BY RANGE (order_date);  
2 CREATE TABLE orders_2025  
    PARTITION OF orders  
    FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

1 RANGE partitioning by date lets Postgres skip entire partitions outside a query's filter.

2 Each partition is a normal table underneath — indexes and constraints still apply.

→ Partitioning is what keeps feature queries fast once your training table passes tens of millions of rows.

Parallel Query Execution

```
1 SET max_parallel_workers_per_gather = 4;  
  
   EXPLAIN SELECT AVG(order_value)  
  
   FROM orders  
  
   WHERE order_date > '2020-01-01';  
  
2 -- Plan shows: Gather -> Parallel Seq Scan
```

1 Raising the worker count lets Postgres split a large scan across CPU cores.

2 A “Gather” node in the plan confirms parallel workers were actually used.

→ Parallel scans give large aggregate-heavy feature queries a near-linear speedup on multi-core machines.

Connection Pooling with PgBouncer

```
[databases]
```

```
ml_portfolio = host=localhost port=5432
```

```
[pgbouncer]
```

```
1 pool_mode = transaction
```

```
max_client_conn = 200
```

```
2 default_pool_size = 20
```

1

transaction mode reuses a connection across many short client requests — ideal for BI tools.

2

A small default_pool_size keeps Postgres's real connection count low and stable.



PgBouncer prevents a live-refreshing Power BI dashboard from exhausting the database's connection limit.

Caching Strategies for Feature Retrieval



Materialized views

Pre-computed results served instantly, refreshed on a schedule.



Query result cache

pg_stat_statements + an app-level cache for repeated dashboard queries.



Partial indexes

Index only the “active” subset of rows that dashboards actually filter.



Read replicas

Offload BI/reporting traffic from the primary transactional database.

07

SECTION 7 OF 9

Exporting PostgreSQL to Power BI

This is where the pipeline becomes visible. A precise, step-by-step procedure for connecting Power BI to Postgres and shipping a live front-end demo.



PostgreSQL → Power BI Integration Overview

PostgreSQL Database (fct_customer_features)

1 ↓ Npgsql / ODBC driver

Power BI Desktop – Get Data → PostgreSQL

2 ↓ Power Query (M) transforms

Data Model – tables, relationships, DAX

↓ publish

3 Power BI Service – scheduled refresh via Gateway

1

The driver translates Power BI's requests into the Postgres wire protocol.

2

Power Query is where light cleanup happens — keep heavy logic in SQL views instead.

3

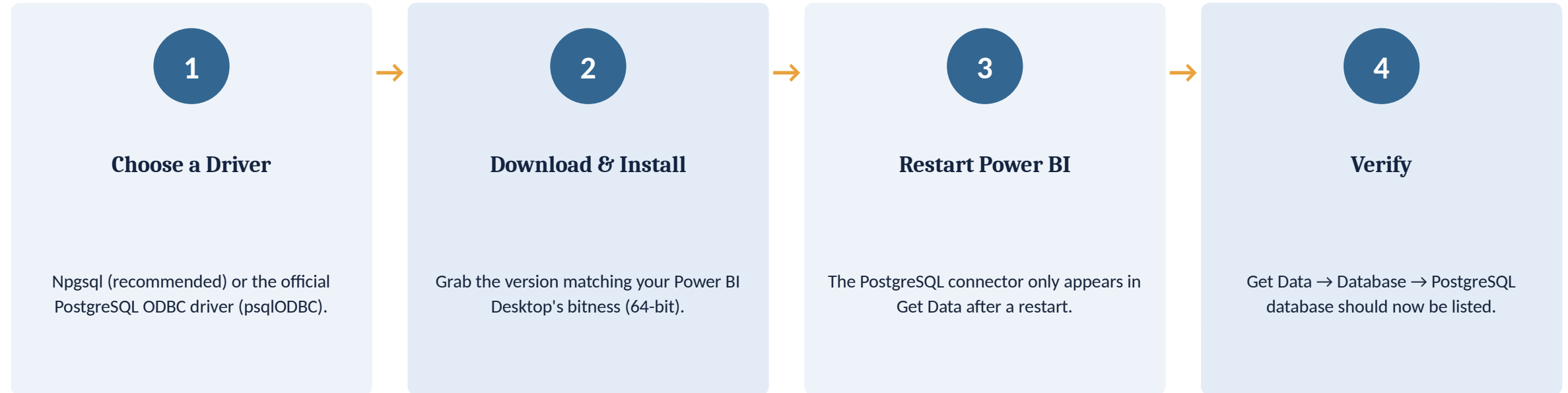
The Gateway is only required for a locally-hosted Postgres instance, not cloud-hosted ones.



Every export issue in this section maps to one link in this chain — know the chain, debug anything.

STEP 1

Installing the PostgreSQL Driver for Power BI



STEP 2

Connecting Power BI to PostgreSQL

Get Data → PostgreSQL database

1 Server: your-host:5432

Database: ml_portfolio

2 Data Connectivity mode: Import | DirectQuery

Enter credentials → Connect

1

Use a read-only role for this connection — never the database's superuser account.

2

This single dropdown decision shapes your entire refresh strategy — next slide.



Store the connection string in a local .env, never hard-coded in a .pbix you commit to GitHub.

Import vs DirectQuery: Which Mode to Use

Import Mode

- ✓ Data cached in-memory — extremely fast to interact with
- ✓ Full DAX feature set available
- ✓ Works offline once loaded
- ✓ Best for portfolio demos with the data already stable

DirectQuery Mode

- ✓ Always shows the current live database state
- ✓ No dataset size limit — queries run against Postgres directly
- ✓ Some DAX functions are restricted
- ✓ Best when the demo needs to prove a “live pipeline” story

Recommendation: use Import for a polished, snappy portfolio demo, DirectQuery when the story is “live data engineering.”

STEP 3

Writing Optimized SQL Views for Power BI

```
1 CREATE VIEW vw_bi_customer_summary AS  
  SELECT c.customer_id, c.plan,  
         f.order_count, f.avg_order_value,  
         f.last_order_date  
  FROM customers c  
2 JOIN fct_customer_features f  
   ON c.customer_id = f.customer_id;
```

1

Expose a clean, purpose-built view — never point Power BI at raw transactional tables.

2

Pre-joining here means Power BI's model doesn't need a slow, wide relationship.



Every column Power BI touches should already be aggregated and typed correctly in the view.

STEP 4

Power Query (M) Transformations

let

1 Source = PostgreSQL.Database("host","db"),
Summary = Source{[Schema="public",
Item="vw_bi_customer_summary"]}[Data],

2 Typed = Table.TransformColumnTypes(Summary,
{{"last_order_date", type date}})

in

Typed

1

PostgreSQL.Database() is the M function created by the driver you installed in Step 1.

2

Type correction here prevents silent date/number mis-parsing later in DAX measures.



Keep M-code thin — filtering and joins belong in the SQL view, not duplicated in Power Query.

Building the Data Model & Relationships

Model Structure

- One fact table: vw_bi_customer_summary
- Dimension tables: dim_date, dim_plan
- Star schema — fact in the center, dimensions radiating out
- Single-direction relationships wherever possible

Why It Matters

- Star schemas make DAX simpler and faster to write
- Fewer bidirectional filters means fewer ambiguous results
- Matches exactly how a hiring manager expects a BI model to look
- Mirrors the feature-store pattern from Section 2

STEP 6

DAX Measures for ML Metrics

1 Avg Order Value :=

```
AVERAGE(vw_bi_customer_summary[avg_order_value])
```

High Value Customers :=

2 CALCULATE(

```
COUNTROWS(vw_bi_customer_summary),
```

```
vw_bi_customer_summary[avg_order_value] > 200
```

```
)
```

1

Simple measures like this feed KPI cards at the top of your dashboard.

2

CALCULATE overrides filter context — the single most important DAX function to master.



Two or three well-chosen DAX measures communicate your model's output far better than a raw table.

Scheduled Refresh & Gateway Setup



Troubleshooting Common Postgres → Power BI Issues

Symptom	Likely Cause	Fix
Connector not listed	Driver not installed / not restarted	Reinstall Npgsql driver, restart Power BI
Connection timeout	Firewall or wrong port/host	Confirm port 5432 is open; check host string
Refresh fails in Service	Gateway not configured	Install & register On-premises Data Gateway
Slow dashboard load	DirectQuery on unindexed view	Add indexes; consider switching to Import
Wrong date formatting	Timezone mismatch	Standardize to UTC in the SQL view, not Power Query



08

SECTION 8 OF 9

Designing the Front-End Demo

A working connection isn't a demo. This section turns the data model into a dashboard that tells a clear, interview-ready story.



Dashboard Design Principles for ML Demos

1

Lead with the answer

Put the key metric or insight top-left — where eyes land first.

2

One page, one question

Each dashboard page should answer a single, clearly-titled question.

3

Show the model's output

Don't just show raw data — visualize predictions, segments, or scores.

4

Consistent visual language

One color per category, reused everywhere — no rainbow charts.

Visualizing Predictions vs Actuals

Recommended Visuals

- Line chart: predicted vs actual revenue over time
- Scatter plot: predicted probability vs outcome
- Bar chart: customers per predicted segment
- KPI cards: model accuracy / R^2 at a glance

What to Avoid

- 3D charts or pie charts with more than 5 slices
- Every column dumped into one giant table
- Unlabeled axes or missing units
- Color palettes with no consistent meaning

Interactive Filters & What-If Parameters



Slicers

Let the viewer filter by plan, date range, or predicted segment live.



Cross-filtering

Clicking one chart filters the others — shows the data model is truly connected.



What-if parameters

A DAX what-if parameter lets viewers adjust a threshold and see segments update.



Drill-through

A summary page that drills into per-customer detail builds real depth.

Storytelling With Your Dashboard in Interviews



Open with the question

“I wanted to know which customers were about to churn.”



Walk the pipeline

Postgres → features → model → Power BI, in that order, briefly.



State the finding

One clear, specific sentence — not “there are interesting patterns.”



End with impact

What decision would this dashboard actually change?

Embedding, Exporting & Sharing Your Report

Sharing Options

- Publish to web (public, unauthenticated viewing)
- Export static PNG screenshots for the README
- Record a 60-second Loom walkthrough
- Embed in a personal portfolio website via iframe

For the GitHub Repo Specifically

- Commit the .pbix file itself (with sample/synthetic data)
- Add 3–5 dashboard screenshots to /docs
- Link the Publish-to-web URL directly in the README
- Never commit real credentials inside the .pbix connection

09

SECTION 9 OF 9

Publishing Your Portfolio Project on GitHub

The final mile: packaging everything we've built into a repository structure that reads as professional in the first ten seconds.



Repository Structure Best Practices

postgres-ml-portfolio/

1 README.md

sql/

2 01_schema.sql

02_algorithms.sql

pipeline/

dashboard/

3 docs/

1 README is the front door — covered in full on the next slide.

2 Numbered SQL files show execution order — the repo can rebuild itself top to bottom.

3 docs/ holds the ERD diagram and dashboard screenshots referenced in the README.

→ A reviewer should understand your entire project's shape from the file tree alone, before opening one file.

Writing a Compelling README.md

```
1 # Customer Churn Risk – Postgres + Power BI
  ## Problem

  ## Data & Schema (link: docs/erd.png)

  ## Methods (SQL algorithms used, w/ links)
2 ## Results (dashboard screenshot + finding)
3 ## How to Run (setup steps)
```

1

A specific, outcome-oriented title beats a generic “SQL Project” every time.

2

Lead the Results section with the dashboard image — visual proof reviewers remember.

3

“How to Run” turns your repo from a display piece into something a reviewer can actually test.



Write the README last, but design it first — every section here maps to a slide in this deck.

Documenting SQL & ER Diagrams

ERD Diagram

- Generate from pgAdmin, dbdiagram.io, or DrawSQL
- Show raw, feature, and dimension layers distinctly
- Export as PNG into /docs and link from the README
- Keep it updated whenever the schema changes

SQL Comments

- A one-line comment above every non-trivial query
- Explain **why** an index or CTE exists, not just what it does
- Group files by pipeline stage: schema, features, algorithms
- Consistent naming: fct_, dim_, vw_ prefixes throughout

CI Checks, Tests & Licensing

Include

- ✓ A LICENSE file (MIT is the standard default choice)
- ✓ A .gitignore excluding credentials and local .env files
- ✓ A simple GitHub Action that lints/validates the SQL files
- ✓ A basic data-quality test script from Section 5

Never Include

- ✗ Real customer data or production credentials
- ✗ Hard-coded database passwords in any script or .pbix
- ✗ Large binary files without Git LFS
- ✗ An unlicensed repo — it signals unfinished work

Badges, GitHub Pages & Project Showcase



README badges

Build status, license, and “made with PostgreSQL” badges add instant credibility.



Pin the repo

Pin it to your GitHub profile — it's the first thing recruiters click.



GitHub Pages

Host a lightweight project summary page directly from the repo, free.



Topics & tags

Tag the repo postgresql, machine-learning, power-bi for discoverability.

Promoting Your Project Beyond GitHub



LinkedIn post

Share the dashboard screenshot with the one-sentence finding as the hook.



Resume line

“Built an end-to-end ML pipeline in PostgreSQL, published live in Power BI.”



Interview talking point

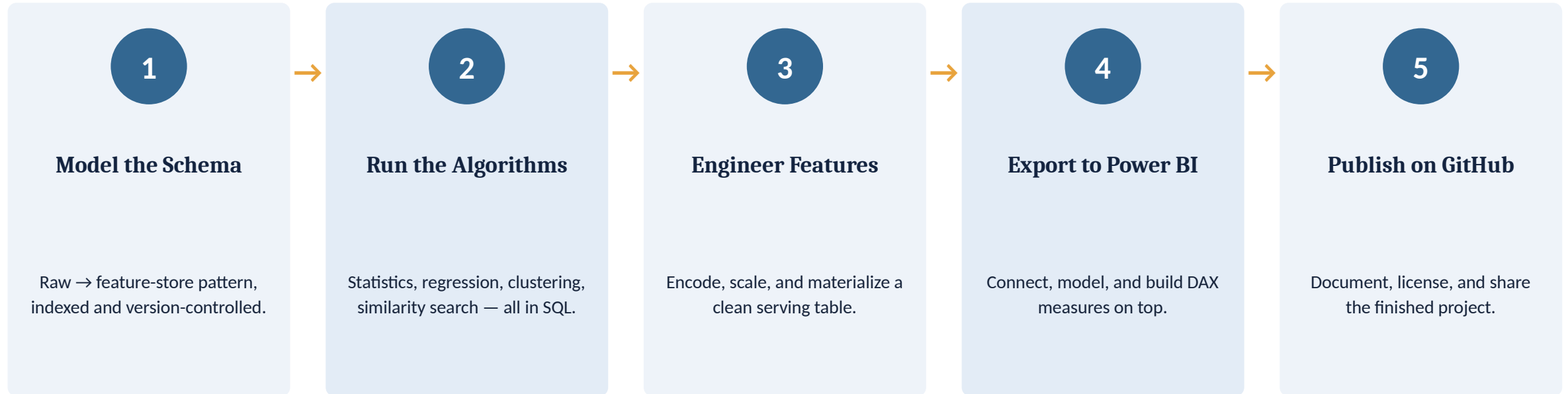
Prepare the 90-second walkthrough from Section 8 in advance.



Community feedback

Post in r/dataengineering or a Discord for real critique before applying.

The Full Pipeline, End to End



Key Takeaways & Common Pitfalls

Remember This

- ✓ Postgres alone covers statistics, regression, clustering & search
- ✓ Feature pipelines belong in versioned SQL, not ad-hoc scripts
- ✓ A materialized view is the bridge between the DB and Power BI
- ✓ The README is graded as strictly as the code

Watch For This

- ✗ Treating the dashboard as an afterthought
- ✗ Skipping data-quality checks before publishing results
- ✗ Leaving commit history as a single “final” commit
- ✗ Choosing a dataset with no real question behind it

THANK YOU

Now Go Build the Project

- ✓ Pick a dataset with a real, specific question behind it
- ✓ Model it with the feature-store pattern from Section 2
- ✓ Implement 3–5 algorithms from Section 4 — not all 12
- ✓ Ship a live Power BI demo and a documented GitHub repo
- ✓ Prepare your 90-second walkthrough before the interview